

```
In [1]: import pandas as pd  
import numpy as np
```

```
In [2]: df = pd.read_csv('C:/Users/ncmia/OneDrive/Desktop/VS_CODE/MCA LAB PROGRAMS/SEM 4
```

```
In [3]: df.head(10)
```

Out[3]:

	age	gender	daily_social_media_hours	platform_usage	sleep_hours	screen_time_bef
--	-----	--------	--------------------------	----------------	-------------	-----------------

0	14.0	NaN	7.9	Instagram	7.4	
---	------	-----	-----	-----------	-----	--

1	19.0	female	1.9	TikTok	8.0	
---	------	--------	-----	--------	-----	--

2	17.0	female	1.3	Instagram	7.6	
---	------	--------	-----	-----------	-----	--

3	15.0	male	7.4	TikTok	6.9	
---	------	------	-----	--------	-----	--

4	15.0	female	NaN	Both	4.9	
---	------	--------	-----	------	-----	--

5	NaN	female	7.4	Both	4.4	
---	-----	--------	-----	------	-----	--

6	18.0	female	2.5	Instagram	6.4	
---	------	--------	-----	-----------	-----	--

7	16.0	male	4.0	Both	4.2	
---	------	------	-----	------	-----	--

8	19.0	female	3.3	TikTok	5.0	
---	------	--------	-----	--------	-----	--

9	15.0	male	1.9	TikTok	4.9	
---	------	------	-----	--------	-----	--



```
In [4]: df.info
```

```

Out[4]: <bound method DataFrame.info of
atform_usage sleep_hours \
0      14.0      NaN      7.9      Instagram      7.4
1      19.0  female      1.9      TikTok      8.0
2      17.0  female      1.3      Instagram      7.6
3      15.0   male      7.4      TikTok      6.9
4      15.0  female      NaN      Both      4.9
...      ...      ...      ...      ...      ...
1195   18.0  female      6.8      Instagram      6.6
1196   16.0   male      2.3      Both      8.0
1197   14.0  female      1.7      Both      8.7
1198   15.0   male      NaN      Both      8.5
1199   16.0      NaN      4.7      TikTok      6.5

      screen_time_before_sleep  academic_performance  physical_activity \
0                        2.9                3.01                1.5
1                        2.9                3.22                0.8
2                        0.5                3.92                0.0
3                        1.6                 NaN                0.8
4                        3.0                2.37                1.4
...                        ...                ...                ...
1195                     NaN                2.76                1.0
1196                     1.9                2.12                0.4
1197                     0.7                3.98                NaN
1198                     2.1                3.19                0.6
1199                     1.0                2.91                0.9

      social_interaction_level  stress_level  anxiety_level  addiction_level \
0                        low                2.0                NaN                1.0
1                        NaN                8.0                1.0               10.0
2                        high                2.0                4.0                2.0
3                        NaN                1.0                7.0                9.0
4                        NaN                3.0                5.0                2.0
...                        ...                ...                ...                ...
1195                     low                3.0                4.0                4.0
1196                     NaN                7.0                4.0                4.0
1197                     high                1.0                1.0                NaN
1198                     high                7.0                9.0                9.0
1199                     medium               5.0                7.0                3.0

      depression_label
0                0.0
1                0.0
2                0.0
3                NaN
4                0.0
...                ...
1195             NaN
1196             0.0
1197             0.0
1198             0.0
1199             0.0

[1200 rows x 13 columns]>

```

```
In [6]: df.dtypes
```

```
Out[6]: age                float64
gender                object
daily_social_media_hours float64
platform_usage        object
sleep_hours           float64
screen_time_before_sleep float64
academic_performance  float64
physical_activity      float64
social_interaction_level object
stress_level           float64
anxiety_level          float64
addiction_level        float64
depression_label       float64
dtype: object
```

The age is a float which is not needed, we have to make it an int value. gender, platform_usage are supposed to be categorical data not objects. social_interaction_level has high, medium and low which can be a ordered category. stress_level, anxiety_level and addiction_level are shown as float because it has nan as a data, we need to convert it to int. depression_label is again should be int as 0 or 1 but since few columns are shown as nan its shown as float. we need to make it int as well

```
In [7]: df['age'] = df['age'].astype('Int64')

df['gender'] = df['gender'].astype('category')
df['platform_usage'] = df['platform_usage'].astype('category')

ordinal_order = pd.CategoricalDtype(categories=['low', 'medium', 'high'], ordered=True)
df['social_interaction_level'] = df['social_interaction_level'].astype(ordinal_order)

df['stress_level'] = df['stress_level'].astype('Int64')
df['anxiety_level'] = df['anxiety_level'].astype('Int64')
df['addiction_level'] = df['addiction_level'].astype('Int64')

df['depression_label'] = df['depression_label'].astype('Int64')

print("Updated dtypes:")
print(df.dtypes)
```

```
Updated dtypes:
age                Int64
gender             category
daily_social_media_hours float64
platform_usage     category
sleep_hours        float64
screen_time_before_sleep float64
academic_performance float64
physical_activity   float64
social_interaction_level category
stress_level        Int64
anxiety_level       Int64
addiction_level     Int64
depression_label    Int64
dtype: object
```

2.

a. descriptive statistics for all numerical values

In [9]: `df.describe()`

Out[9]:

	age	daily_social_media_hours	sleep_hours	screen_time_before_sleep	acad...
count	1105.0	1056.000000	1008.000000	1140.000000	
mean	15.901357	4.544129	6.443155	1.735965	
std	2.016294	2.027443	1.457104	0.717721	
min	13.0	1.000000	4.000000	0.500000	
25%	14.0	2.800000	5.100000	1.100000	
50%	16.0	4.500000	6.400000	1.700000	
75%	18.0	6.300000	7.700000	2.325000	
max	19.0	8.000000	9.000000	3.000000	

In [10]: `print("Variable with highest variation (std):", df.describe().loc['std'].idxmax())`

Variable with highest variation (std): stress_level

b. mean and median between daily_social_media_hours & sleep_hours column

In [12]: `import matplotlib.pyplot as plt`

`fig, axes = plt.subplots(1, 2, figsize=(12, 4))`

`for ax, col in zip(axes, ['daily_social_media_hours', 'sleep_hours']):`

`data = df[col].dropna()`

`mean_val = data.mean()`

`median_val = data.median()`

`ax.hist(data, bins=20, color='steelblue', edgecolor='white')`

`ax.axvline(mean_val, color='red', linestyle='--', linewidth=2, label=f'`

`ax.axvline(median_val, color='orange', linestyle='--', linewidth=2, label=f'`

`ax.set_title(col)`

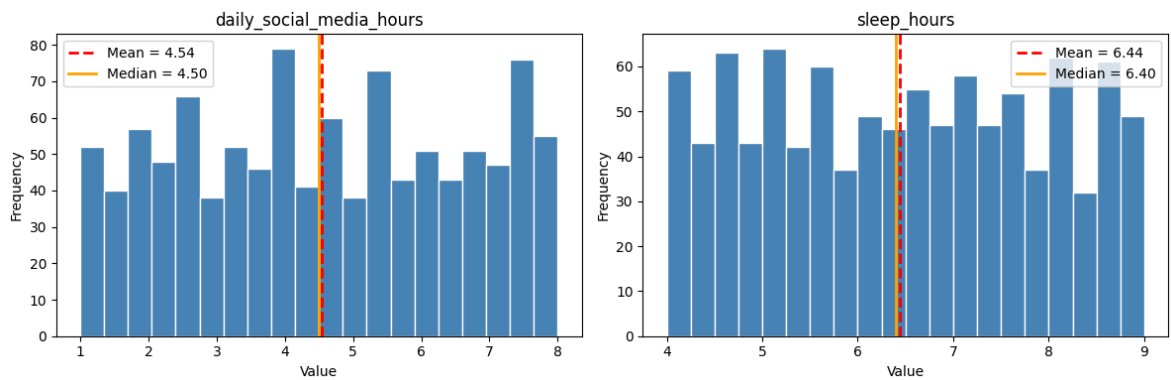
`ax.set_xlabel('Value')`

`ax.set_ylabel('Frequency')`

`ax.legend()`

`plt.tight_layout()`

`plt.show()`



This shows that both the columns have a normal skewness because the mean and median are almost at the same place. Hence we can say that these columns are normally distributed.

3.

a. indentigy missing values

```
In [13]: missing_count = df.isnull().sum()
missing_percent = (df.isnull().sum() / len(df)) * 100

missing_df = pd.DataFrame({'Missing Count': missing_count, 'Missing %': missing_percent})
print(missing_df[missing_df['Missing Count'] > 0])
```

	Missing Count	Missing %
age	95	7.92
gender	228	19.00
daily_social_media_hours	144	12.00
platform_usage	168	14.00
sleep_hours	192	16.00
screen_time_before_sleep	60	5.00
academic_performance	120	10.00
physical_activity	108	9.00
social_interaction_level	168	14.00
stress_level	108	9.00
anxiety_level	108	9.00
addiction_level	72	6.00
depression_label	192	16.00

b.

for daily_social_media_hours, sleep_hours and screen_time_before_sleep are all continuous numerical columns so we can use mean or median.

for daily_social_media_hours and screen_time_before_sleep we should use median imputation because some teens might use screens a lot more than others which creates outliers, so median is safer here.

for sleep_hours we can use mean imputation.

```
In [14]: df['daily_social_media_hours'].fillna(df['daily_social_media_hours'].median(), inplace=True)
df['sleep_hours'].fillna(df['sleep_hours'].mean(), inplace=True)
df['screen_time_before_sleep'].fillna(df['screen_time_before_sleep'].median(), inplace=True)
```

```
print("Missing values after imputation:")
print(df[['daily_social_media_hours', 'sleep_hours', 'screen_time_before_sleep']])
```

Missing values after imputation:

```
daily_social_media_hours    0
sleep_hours                 0
screen_time_before_sleep    0
dtype: int64
```

C:\Users\ncmia\AppData\Local\Temp\ipykernel_21180\1789112958.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['daily_social_media_hours'].fillna(df['daily_social_media_hours'].median(),
inplace=True)
```

C:\Users\ncmia\AppData\Local\Temp\ipykernel_21180\1789112958.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['sleep_hours'].fillna(df['sleep_hours'].mean(), inplace=True)
```

C:\Users\ncmia\AppData\Local\Temp\ipykernel_21180\1789112958.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['screen_time_before_sleep'].fillna(df['screen_time_before_sleep'].median(),
inplace=True)
```

c.

for stress_level, anxiety_level and addiction_level all values are on a scale of 1 to 10 so for all three of them we use median to fill in empty cells.

```
In [15]: df['stress_level'].fillna(df['stress_level'].median(), inplace=True)
df['anxiety_level'].fillna(df['anxiety_level'].median(), inplace=True)
df['addiction_level'].fillna(df['addiction_level'].median(), inplace=True)
```

```
print("Missing values after imputation:")
print(df[['stress_level', 'anxiety_level', 'addiction_level']].isnull().sum())
```

Missing values after imputation:

```
stress_level      0
anxiety_level     0
addiction_level   0
dtype: int64
```

C:\Users\ncmia\AppData\Local\Temp\ipykernel_21180\810611801.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['stress_level'].fillna(df['stress_level'].median(), inplace=True)
```

C:\Users\ncmia\AppData\Local\Temp\ipykernel_21180\810611801.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['anxiety_level'].fillna(df['anxiety_level'].median(), inplace=True)
```

C:\Users\ncmia\AppData\Local\Temp\ipykernel_21180\810611801.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['addiction_level'].fillna(df['addiction_level'].median(), inplace=True)
```

d.

If teens with lower sleep_hours or higher daily_social_media_hours are more likely to skip answering the anxiety question or other mental health questions because they could be distracted while giving the data, then the missingness is related to other observed variables and so it is MAR.

4.

a. distribution of gender, platform_usage and depression_label

```
In [16]: import matplotlib.pyplot as plt

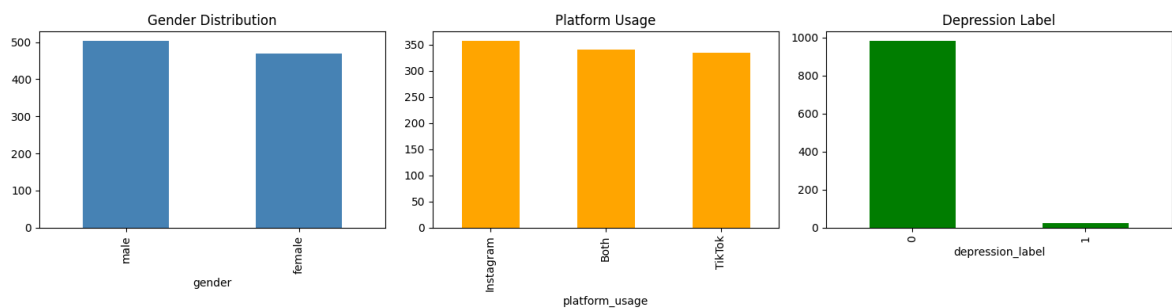
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

df["gender"].value_counts().plot(kind="bar", ax=axes[0], color="steelblue")
axes[0].set_title("Gender Distribution")

df["platform_usage"].value_counts().plot(kind="bar", ax=axes[1], color="orange")
axes[1].set_title("Platform Usage")

df["depression_label"].value_counts().plot(kind="bar", ax=axes[2], color="green")
axes[2].set_title("Depression Label")

plt.tight_layout()
plt.show()
```



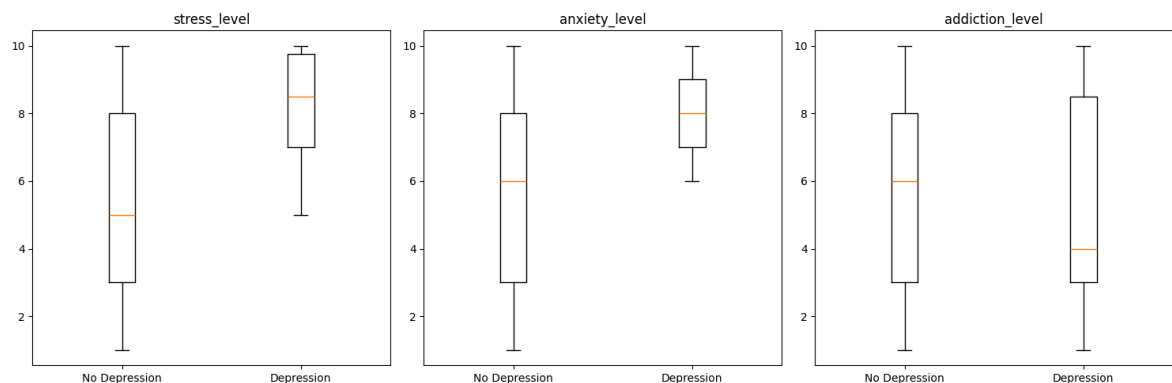
b. boxplots of stress_level, anxiety_level and addiction_level grouped by depression_label

```
In [17]: df_dep = df.dropna(subset=["depression_label"])

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

for ax, col in zip(axes, ["stress_level", "anxiety_level", "addiction_level"]):
    data0 = df_dep[df_dep["depression_label"] == 0][col].dropna()
    data1 = df_dep[df_dep["depression_label"] == 1][col].dropna()
    ax.boxplot([data0, data1], labels=["No Depression", "Depression"])
    ax.set_title(col)

plt.tight_layout()
plt.show()
```



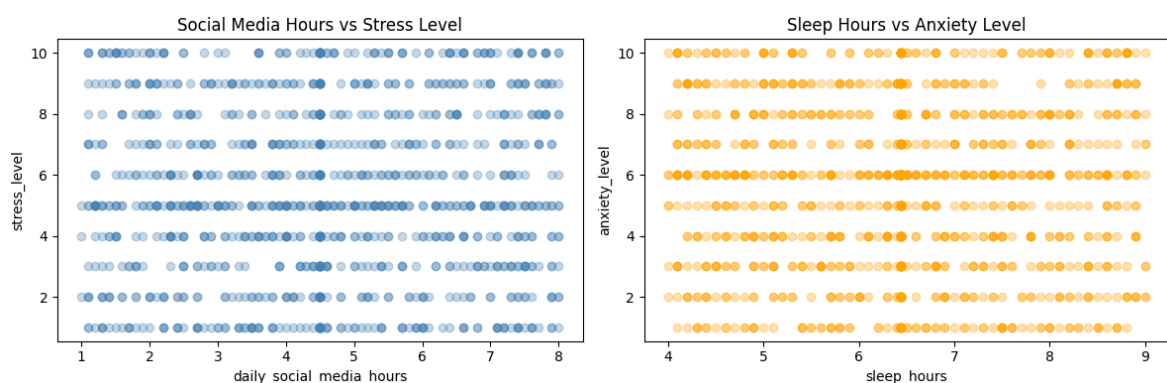
c. scatter plots between daily_social_media_hours vs stress_level and sleep_hours vs anxiety_level

```
In [18]: fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].scatter(df["daily_social_media_hours"], df["stress_level"], alpha=0.3, c
axes[0].set_xlabel("daily_social_media_hours")
axes[0].set_ylabel("stress_level")
axes[0].set_title("Social Media Hours vs Stress Level")

axes[1].scatter(df["sleep_hours"], df["anxiety_level"], alpha=0.3, color="orange
axes[1].set_xlabel("sleep_hours")
axes[1].set_ylabel("anxiety_level")
axes[1].set_title("Sleep Hours vs Anxiety Level")

plt.tight_layout()
plt.show()
```



d. Chart 1 - in gender distribution the bar chart shows that there are more male teens than female teens in this dataset. this means the dataset is slightly imbalanced by gender.

Chart 2 - in depression label of stress level, the bar chart shows that most teens in the dataset do not have depression and only a very small number have depression. this tells us the dataset is highly imbalanced for the depression label.

Chart 3 - for sleep hours and anxiety level, the scatter plot shows that teens with less sleep may have anxiety levels spread across all values, but teens with more sleep hours generally show lower anxiety.

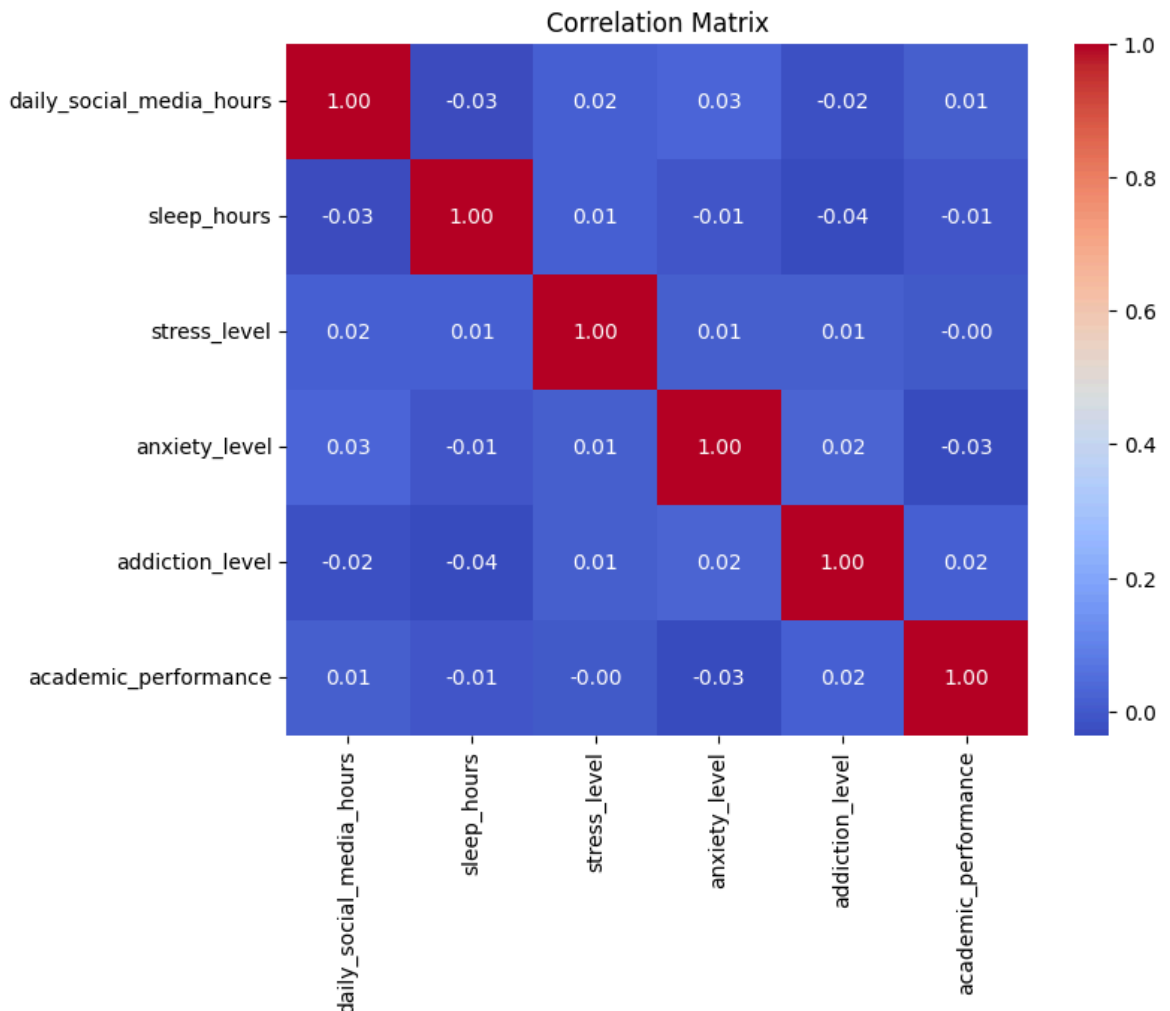
5.

```
In [19]: import seaborn as sns

cols = ["daily_social_media_hours", "sleep_hours", "stress_level", "anxiety_level"]

corr = df[cols].corr()

plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```



stress_level, anxiety_level and addiction_level are strongly positively correlated with each other which means teens who are stressed also tend to have higher anxiety and addiction.

daily_social_media_hours has a positive correlation with stress and anxiety meaning more social media usage is connected to higher stress and anxiety.

sleep_hours has a slight negative correlation with stress meaning teens who sleep more tend to have lower stress.

academic_performance has a slight negative correlation with stress and anxiety showing that higher stress tends to lower academic performance.

categorical variables should not be directly included in correlation analysis because correlation works on numerical values only. we need to encode them first using label encoding or one hot encoding before including them.

6.

a. IQR method to identify outliers

```
In [20]: cols = ["daily_social_media_hours", "sleep_hours", "stress_level", "anxiety_level", "addiction_level", "academic_performance"]
for col in cols:
```

```

Q1 = df[col].quantile(0.25)
Q3 = df[col].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers = df[(df[col] < lower) | (df[col] > upper)]
print(f"{col}: lower={lower:.2f}, upper={upper:.2f}, outliers found={len(out

```

daily_social_media_hours: lower=-1.50, upper=10.50, outliers found=0

sleep_hours: lower=2.40, upper=10.40, outliers found=0

stress_level: lower=-4.50, upper=15.50, outliers found=0

anxiety_level: lower=-4.50, upper=15.50, outliers found=0

b. extreme values in social media usage are not necessarily data errors. some teens genuinely spend 7 to 8 hours per day on social media especially for entertainment, socialising or studying. so these high values represent real behavioral patterns of heavy users. since the column max is 8 hours and the scale seems fixed between 1 and 8, there are no values outside a realistic range. so we treat these extreme values as meaningful patterns rather than errors.

7.

a. from the correlation matrix we can see that daily_social_media_hours has a positive relationship with stress_level, anxiety_level and addiction_level. this means teens who spend more time on social media tend to have higher stress, anxiety and addiction. so heavy social media usage is linked to poorer mental health outcomes.

b. teens with less sleep and more screen time before sleep tend to have higher stress and anxiety levels. this shows that poor sleep habits like sleeping late due to phone usage have a negative impact on mental health. reducing screen time before bed and getting more sleep can help improve mental health.

c. from the boxplots and correlation analysis, high stress_level, anxiety_level and addiction_level are the main factors associated with depression in teens. teens with depression tend to have much higher values in all three of these compared to teens without depression.

d. teens with depression tend to have lower academic_performance and lower physical_activity compared to teens without depression. their social_interaction_level also tends to be low. this shows that depression negatively affects how teens perform in studies, how active they are physically and how much they interact with others socially.